

<https://www.sqldbachamps.com>

<https://github.com/PMSQLDBA/PraveenMadupu>

Telegram Group → SQLDBACloud-DevOps <https://t.me/+r2OYsT2qRZJkYWVI>

Praveen Madupu

Mb: +91 98661 30093

Database Administrator

## What is SQL Server Configuration Manager (SSCM)?

**SQL Server Configuration Manager** is a Microsoft Management Console (MMC) snap-in tool used to manage the **SQL Server services**, **network protocols**, and **client connectivity settings** for SQL Server instances.

 **It's not a standalone EXE — it's part of SQL Server tools**, and it directly edits the **Windows Registry** safely to apply SQL Server-related configurations.

## Why Use Configuration Manager (Not Services.msc or Regedit)?

Although you *can* start/stop services using services.msc, **Configuration Manager is preferred** because:

- It **updates service parameters** (like startup accounts, passwords, ports) directly in the **registry** in a supported way.
- It **refreshes SQL Server's internal configurations** automatically.
- It prevents **corruption or startup failures** that can occur when using normal Windows tools.

## How to Open SQL Server Configuration Manager

Depending on your version:

| SQL Server Version | Open Command           |
|--------------------|------------------------|
| SQL Server 2019    | SQLServerManager15.msc |
| SQL Server 2017    | SQLServerManager14.msc |
| SQL Server 2016    | SQLServerManager13.msc |
| SQL Server 2014    | SQLServerManager12.msc |
| SQL Server 2012    | SQLServerManager11.msc |

Or search in Windows:

 **Start → SQL Server Tools → SQL Server Configuration Manager**

## Main Components in SQL Server Configuration Manager

### 1 SQL Server Services

Manages all SQL-related services installed on the machine.

**Includes:**

- **SQL Server (MSSQLSERVER or InstanceName)** → Main Database Engine.
- **SQL Server Agent** → Job Scheduler and Automation.
- **SQL Server Browser** → Helps clients locate named instances.
- **SQL Server Reporting Services (SSRS)** → For reports (if installed).
- **SQL Server Integration Services (SSIS)** → For ETL/Data Integration.

**Key Actions DBAs perform:**

- Start/Stop/Restart services.
- Change **service accounts** and passwords securely.
- Configure **Startup Type**: Automatic / Manual / Disabled.
- Check **service status** after patching or reboot.

### **Real-time Scenario Example:**

 *After applying a Windows Service Account password change policy, you update the password for the SQL Server and Agent services through Configuration Manager — not through services.msc — to ensure registry & permissions sync.*

## 2 SQL Server Network Configuration (for each instance)

Controls **server-side network protocols** that SQL Server listens on.

**Protocols you'll see:**

- **Shared Memory** → Local connections only (default enabled).
- **Named Pipes** → For legacy or internal apps (optional).
- **TCP/IP** → For remote connections (must be enabled).
- **VIA (Virtual Interface Adapter)** → Deprecated, rarely used.

**You can:**

- Enable/Disable each protocol.
- Configure TCP/IP **port numbers**.
- Specify **IP Addresses, TCP Dynamic Ports, or Static Ports**.

**Real-time Scenario Example:**

◆ *A developer cannot connect to SQL Server remotely.*

You open SSCM → SQL Server Network Configuration → Enable TCP/IP → Set static port (e.g., 1433) → Restart SQL Service → Test remote connectivity using telnet servername 1433.

## 3 SQL Native Client Configuration

Controls **client-side network settings** used by applications that connect to SQL Server.

It has two parts:

- **Client Protocols**
- **Aliases**

◆ **Client Protocols**

- Controls **protocol order** used by clients (TCP/IP, Named Pipes, Shared Memory).
- You can **disable** unused ones for security.

◆ **Aliases**

- Allows you to **map a custom name** to a SQL Server instance.

**Example:**

Alias name = ProdSQL

Actual server = SQLPROD01\INST01

Protocol = TCP/IP

Port = 51433

Now, applications can connect using just ProdSQL.

**Real-time Scenario Example:**

◆ *During DR (Disaster Recovery), the application must point to a secondary SQL Server.*

Instead of modifying the app connection string, you change the **client alias** in SSCM to point to the new server.

## ⚙️ Registry Paths Controlled by Configuration Manager

When you use SSCM, it updates these registry locations safely:

HKEY\_LOCAL\_MACHINE\SOFTWARE\Microsoft\Microsoft SQL Server\<InstanceID>\MSSQLServer\Parameters

HKEY\_LOCAL\_MACHINE\SYSTEM\CurrentControlSet\Services\MSSQLSERVER

HKEY\_LOCAL\_MACHINE\SOFTWARE\Microsoft\MSSQLServer\Client\

⚠️ **Do NOT modify these directly** — always use SSCM.

## Real-World Scenarios for SQL Server Configuration Manager

| Scenario                                                                                                              | Action in SSCM                                                                 |
|-----------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------|
|  Service account password expired    | Update password for SQL Service & SQL Agent                                    |
|  Remote connections failing          | Enable TCP/IP and check static port                                            |
|  AlwaysOn setup                      | Ensure SQL Server service runs under domain account and has proper permissions |
|  Port blocked by firewall            | Configure static port (1433 or custom) and open in firewall                    |
|  Failover cluster node configuration | Start/Stop SQL service safely on one node                                      |
|  DR failover alias switch            | Modify client alias mapping to DR instance                                     |
|  Multi-instance configuration        | Manage protocols and ports for each named instance separately                  |

## Differences Between SSCM and SSMS

| Feature     | SQL Server Configuration Manager                  | SQL Server Management Studio (SSMS)              |
|-------------|---------------------------------------------------|--------------------------------------------------|
| Scope       | Windows-level config (services, ports, protocols) | Database-level config (logins, backups, queries) |
| Works on    | Registry + Services                               | SQL Engine & Metadata                            |
| Used for    | Service control, connectivity, network tuning     | Query execution, DB admin, security              |
| Permissions | Requires Windows Admin rights                     | Requires SQL login rights                        |

## Best Practices for Using SQL Server Configuration Manager

- ✅ Always use SSCM to change:
  - Service accounts and passwords
  - Network protocol settings
  - Startup parameters
- ✅ After making changes:
  - **Restart the service** for changes to take effect.
  - **Document all changes** (especially ports, service accounts).
  - **Check error logs** if SQL Server fails to start.
- ✅ Disable unnecessary protocols (e.g., Named Pipes) for **security hardening**.

## Quick Real-Time Example Walkthrough

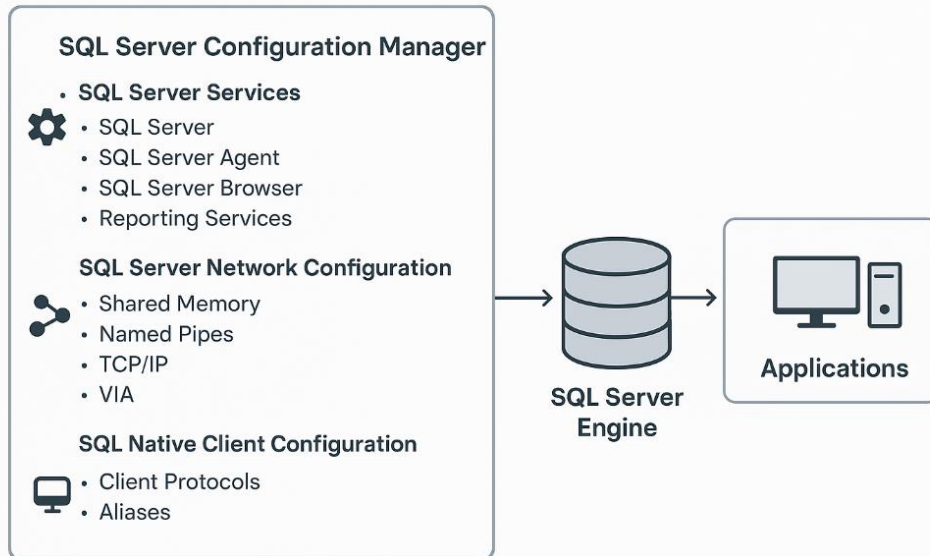
### Scenario:

A remote app cannot connect to your SQL Server instance SQL01\PROD from another network.

### Troubleshooting Steps:

1. Open **SQL Server Configuration Manager**.
2. Navigate to → **SQL Server Network Configuration** → **Protocols for PROD**.
3. Ensure **TCP/IP = Enabled**.
4. Double-click **TCP/IP** → **IP Addresses** tab.
  - Under IPAll, set:
  - TCP Dynamic Ports = (blank)
  - TCP Port = 1433
5. Restart **SQL Server (PROD)** service.
6. On the firewall → Allow inbound 1433.
7. Test connection using:
8. telnet SQL01 1433
9. Connection works ✅

Visual diagram showing the *SQL Server Configuration Manager*:



<https://www.sqldbachamps.com/>

## 1. What is SQL Server Network Configuration?

**SQL Server Network Configuration** defines **how clients connect** to your SQL Server instance over the network.

It controls:

- Which network protocols SQL Server will listen on
- The ports used for communication
- How SQL Server accepts local vs remote connections
- Encryption, packet size, and alias configurations

You configure this mainly using **SQL Server Configuration Manager (SSCM)** under:

**SQL Server Network Configuration → Protocols for [InstanceName]**

## 2. SQL Server Network Stack Overview

When a client connects, here's what happens conceptually:

Client Application



SQL Server Client Library (e.g., ODBC, OLE DB, .NET SqlClient)



SQL Server Network Library (SNI Layer)



Chosen Protocol (TCP/IP, Named Pipes, Shared Memory)



SQL Server Listener (sqlservr.exe)

Each **protocol** has its own method of communication.

## 3. Major Components Inside SQL Server Network Configuration

### Shared Memory

- **Local-only protocol** (works within the same machine)
- Fastest connection (no network stack)
- Uses OS shared memory segments
- **Default enabled**

 **Use case:** Testing connectivity on the same host

 **Limitation:** Not usable for remote connections

 **Real-time scenario:**

You connect via SSMS using (local) or . or localhost — this uses Shared Memory.

### Named Pipes

- Uses **Windows Named Pipe protocol** (\\.\pipe\MSSQL\$InstanceName\sql\query)
- Can work over a network, but slower than TCP/IP.
- Mainly used for **legacy applications** or **intranet setups**.

 **Use case:** When TCP/IP is blocked but inter-server communication within domain is needed

 **Limitation:** Slower and can have firewall permission issues

 **Real-time scenario:**

You're troubleshooting connectivity and find TCP/IP disabled — you can temporarily enable Named Pipes to verify network-level access.

### TCP/IP

- **Primary network protocol** for SQL Server.
- Communicates via **TCP sockets** on a defined **port** (default: 1433).
- Supports both IPv4 and IPv6.

 **Use case:** Standard for all remote SQL Server connections

 **Limitation:** Requires correct port/firewall/DNS configuration

 **Real-time scenario:**

Application fails to connect:

“Cannot connect to SQL Server — Error 26 or 10061.”

You verify that TCP/IP is disabled or port 1433 is blocked.

#### **4 VIA (Virtual Interface Adapter)**

- Legacy, now **deprecated**.
- Used for high-speed SAN or clustering interconnects in older setups.

 No longer supported in modern SQL Server versions.

#### **5 SQL Native Client / SNI Layer**

- The **SQL Server Network Interface (SNI)** layer handles client communication.
- It negotiates encryption, packet size, and protocol selection.

 **Key for troubleshooting** when certain clients fail to connect even if protocols are enabled.

#### **4. TCP/IP Details**

When you open **TCP/IP properties** in Configuration Manager:

- **Protocol Enabled:** Yes/No
- **Listen All:** If Yes, SQL listens on all IPs on the host
- **IP Addresses tab:**
  - **Active/Enabled:** Per-IP control
  - **TCP Port:** Static port (1433 for default instance)
  - **TCP Dynamic Ports:** Used for named instances

#### **Named vs Default Instance Behavior**

| Type             | Protocol | Port                              |
|------------------|----------|-----------------------------------|
| Default Instance | TCP/IP   | 1433 (static)                     |
| Named Instance   | TCP/IP   | Dynamic (assigned by SQL Browser) |

#### **SQL Browser Service**

- Listens on **UDP port 1434**
- Responds to client queries telling them which **port** a named instance is listening on.

 **Real-time scenario:**

If SQL Browser is stopped, named instance connections fail unless you use:

ServerName,PortNumber (e.g., SQLPROD01,51432)

#### **5. Tools for Configuration and Testing**

| Tool                             | Purpose                             |
|----------------------------------|-------------------------------------|
| SQL Server Configuration Manager | Enable/disable protocols, set ports |
| SQL Server Error Log             | Shows listening ports               |
| **netstat -ano                   | findstr 1433**                      |
| telnet servername 1433           | Test port accessibility             |
| ping servername                  | Test DNS resolution                 |
| SQLCMD                           | Command-line connection test        |
| PortQry.exe                      | Advanced port testing               |

| Tool                            | Purpose                     |
|---------------------------------|-----------------------------|
| PowerShell (Test-NetConnection) | Verify network connectivity |

## 6. Real-Time Scenarios (Production Examples)

### Scenario 1 — Application can't connect remotely

#### Error:

A network-related or instance-specific error occurred while establishing a connection to SQL Server.

#### Checks:

1. TCP/IP enabled?
2. Firewall allows 1433 inbound?
3. SQL Browser running (for named instance)?
4. SQL Server service bound to correct IP?
5. Use telnet or Test-NetConnection to validate.

### Scenario 2 — Named instance not connecting

**Root cause:** SQL Browser stopped.

#### Fix:

- Start SQL Browser service.
- Or connect explicitly: SQLPROD01,51432.

### Scenario 3 — Port conflict after patching

**Root cause:** Another app already using 1433.

#### Fix:

- Assign static alternate port (e.g., 1444).
- Update app connection string: SQLPROD01,1444.

### Scenario 4 — Connectivity fine locally, fails remotely

**Root cause:** Shared Memory works locally but TCP/IP disabled or blocked remotely.

#### Fix:

- Enable TCP/IP in Configuration Manager.
- Restart SQL Service.

### Scenario 5 — Encryption requirement

**Root cause:** Security policy requires encryption in transit.

#### Fix:

- Configure **Force Encryption = Yes** in Configuration Manager.
- Install valid SSL/TLS certificate in SQL Server.
- Restart service.

### Scenario 6 — Multi-NIC Servers

**Root cause:** SQL Server listening on the wrong IP.

#### Fix:

- In TCP/IP properties → IP Addresses tab → Disable unneeded IPs.
- Set "Listen All" = No, then explicitly configure the required IP.

## 7. DBA Best Practices

-  Enable only required protocols (usually TCP/IP + Shared Memory).
-  Use **static port** for better firewall and monitoring.
-  Regularly test connectivity using scripts or PowerShell.
-  Monitor the **SQL Browser service** for named instances.

- ✓ Document instance port mappings.
- ✓ Restrict access to SQL ports via firewalls / VPNs.
- ✓ Use **encryption (TLS)** for compliance environments.
- ✓ Audit failed logins and connectivity issues in SQL Logs.

## 🔧 8. Useful SQL Queries

Check which protocols SQL Server is using:

```
SELECT protocol_desc, type_desc, local_net_address, local_tcp_port
```

```
FROM sys.dm_exec_connections
```

```
WHERE session_id = @@SPID;
```

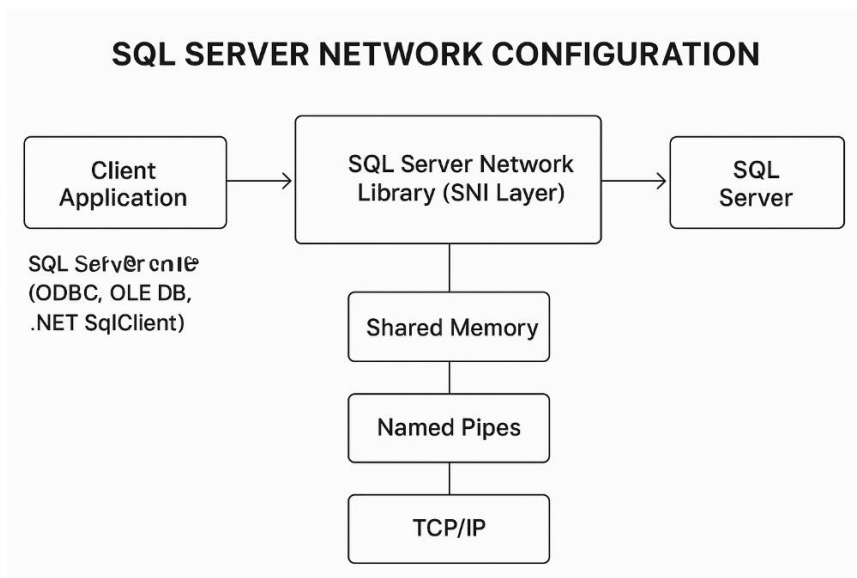
Check if encryption is enabled:

```
SELECT encrypt_option
```

```
FROM sys.dm_exec_connections
```

```
WHERE session_id = @@SPID;
```

**Visual network configuration diagram** showing how client, SQL Browser, and protocols interact:





**SQL Native Client Configuration** is *critical* for SQL Server DBAs, especially when dealing with **connectivity troubleshooting**, **linked servers**, **application connections**, and **network optimization** in real-time production environments.

### ◆ 1. What is SQL Native Client?

**SQL Native Client (SNAC)** is a **standalone data access library** used by client applications to connect to SQL Server.

It includes both:

- **ODBC (Open Database Connectivity) provider**, and
- **OLE DB (Object Linking and Embedding Database) provider**

✚ It acts as a bridge between your **application** and **SQL Server**.

### ◆ 2. Where Do You Find SQL Native Client Configuration?

In **SQL Server Configuration Manager** (on client or server):

Start → SQL Server Configuration Manager → SQL Native Client Configuration

You'll see two main components:

1. **Client Protocols**
2. **Aliases**

### ◆ 3. Components of SQL Native Client Configuration

#### 🧠 A. Client Protocols

These define *how* your client connects to SQL Server — i.e., which network library or protocol it uses.

Common protocols include:

| Protocol             | Description                                               | Default  | Use Case                                                          |
|----------------------|-----------------------------------------------------------|----------|-------------------------------------------------------------------|
| <b>Shared Memory</b> | Local connections only (same machine).                    | Enabled  | Fastest for local DB access (e.g., SSMS connecting to localhost). |
| <b>TCP/IP</b>        | Used for remote connections via IP address or hostname.   | Enabled  | Most common in production; supports encryption, firewall config.  |
| <b>Named Pipes</b>   | Uses named pipe over SMB<br>(\ServerName\pipe\sql\query). | Disabled | Legacy apps or intranet networks.                                 |
| <b>VIA</b>           | Virtual Interface Adapter (deprecated).                   | Disabled | Used in special SAN environments (now obsolete).                  |

#### ⚙️ B. Aliases

Aliases let you **map a custom name** to a specific SQL Server instance, IP, or port.

#### ◆ Example:

If your application connects to SQLPROD, but your real server name is PROD-SQL01,1433, you can create an alias:

Alias Name: SQLPROD

Server: PROD-SQL01

Port: 1433

Protocol: TCP/IP

Then, in your app's connection string:

Server=SQLPROD;Database=SalesDB;Trusted\_Connection=yes;

➡ It will actually connect to PROD-SQL01,1433.

### ✅ Real-time scenario use:

- During **data center migration**, you move databases to a new server but don't want to update connection strings in hundreds of apps — you just update the alias.

### ◆ 4. SQL Native Client Configuration Paths

There are **two levels**:

1. **32-bit configuration**
2. SQL Native Client Configuration (32-bit)

Used by 32-bit apps or tools (e.g., old SSIS packages, 32-bit Excel, etc.)

3. **64-bit configuration**
4. SQL Native Client Configuration

Used by 64-bit apps or tools.

📁 They are separate, so sometimes you may need to configure aliases in *both*.

### ◆ 5. SQL Native Client Versions

Each SQL Server version had its own SNAC build:

#### SQL Server Version Native Client DLL Example DLL Name

|                 |           |               |
|-----------------|-----------|---------------|
| SQL Server 2005 | SQLNCLI   | sqlncli.dll   |
| SQL Server 2008 | SQLNCLI10 | sqlncli10.dll |
| SQL Server 2012 | SQLNCLI11 | sqlncli11.dll |

⚠ From SQL Server 2016 onwards, **SNAC is deprecated**.

You should use **Microsoft OLE DB Driver for SQL Server (MSOLEDBSQL)** or **ODBC Driver for SQL Server (MSODBCSQL)**.

### ◆ 6. How SQL Native Client Works (Flow Diagram)

Application → SQL Native Client → Network Protocol (TCP/IP / Named Pipes / Shared Memory) → SQL Server

Example:

1. Application sends a connection request.
2. SQL Native Client checks the alias.
3. Chooses an available protocol (TCP/IP by default).
4. Sends the request to SQL Server.
5. Establishes session + authentication.

### ◆ 7. Realtime Scenarios for DBAs

#### 🌿 Scenario 1: Application Cannot Connect After Migration

**Symptoms:**

- Error: Cannot connect to SQLPROD
- Old alias points to old IP.

**Fix:**

Update **SQL Native Client Alias** to new server IP:

SQL Native Client Configuration → Aliases → Edit SQLPROD → Change IP/Port

✅ No need to reconfigure all apps.

#### 🌿 Scenario 2: SQL Connection is Slow Over Network

**Symptoms:**

- Local connections are fast, remote ones are slow.

**Checks:**

1. Ensure **TCP/IP is enabled**.

2. Verify **TCP Port (1433)** is open in firewall.
3. Disable **Named Pipes** if not used (as fallback may delay connection).
4. Force TCP/IP protocol in connection string:
5. tcp:SQLSERVER,1433

### Scenario 3: Connection Fails Intermittently

#### Possible Cause:

DNS resolution issue or SQL Browser service not running.

#### Solution:

- Create a **TCP alias** with static port.
- Use:
- Alias Name: SQLPROD
- Server: 192.168.10.15
- Protocol: TCP/IP
- Port: 1433

### Scenario 4: Coexistence of Multiple SQL Versions

#### Example:

- You have SQL 2008 (uses SQLNCLI10) and SQL 2016 (uses SQLNCLI11).
- Your legacy app uses the old SNAC, and a new app uses MSOLEDBSQL.

#### Fix:

Install both clients, configure each with the correct protocol and alias.

### Scenario 5: SSIS Package Fails on Another Server

#### Cause:

Package uses SQLNCLI11 provider, but destination server only has SQLNCLI10 installed.

#### Solution:

Install **SQL Server Native Client 11** or update the **connection manager** to use an existing driver (ODBC / MSOLEDBSQL).

## 8. Verification & Troubleshooting Commands

#### Check Installed Providers:

Get-OdbcDriver | Where-Object Name -like "\*SQL\*"

#### Check Alias via Registry:

HKEY\_LOCAL\_MACHINE\SOFTWARE\Microsoft\MSSQLServer\Client\ConnectTo

#### Test Connection:

sqlcmd -S AliasName -E

## 9. Real-Time Best Practices for SQL Native Client

-  Always **use TCP/IP** for production (stable and fast).
-  **Set static ports** for named instances (to avoid SQL Browser dependency).
-  For **migration**, rely on **aliases** — easy redirection.
-  Keep **drivers updated** — newer drivers handle TLS 1.2 / encryption better.
-  Use **MSOLEDBSQL** or **ODBC** for new applications.
-  Always **test connectivity** from both local and remote systems.

## ◆ 10. Migration Path (SNAC → ODBC/OLE DB)

Since SNAC is deprecated:

| Old SNAC  | New Equivalent      |
|-----------|---------------------|
| SQLNCLI11 | MSOLEDBSQL (OLE DB) |
| SQLNCLI11 | MSODBCSQL (ODBC)    |

Example connection string update:

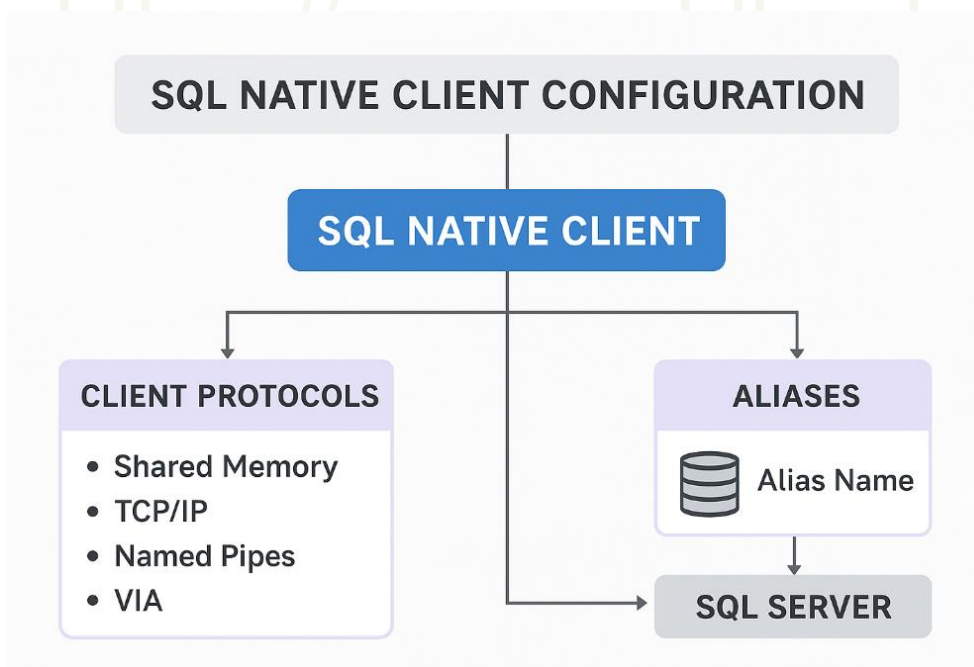
[Old] → Provider=SQLNCLI11;Server=MyServer;Database=DB1;Trusted\_Connection=Yes;

[New] → Provider=MSOLEDBSQL;Server=MyServer;Database=DB1;Trusted\_Connection=Yes;

## ✅ Summary

| Component             | Purpose                                                    | Realtime Use                             |
|-----------------------|------------------------------------------------------------|------------------------------------------|
| Client Protocols      | Determines how clients connect (TCP/IP, Named Pipes, etc.) | Network optimization, troubleshooting    |
| Aliases               | Custom name mapping to servers                             | App migration, DNS independence          |
| SNAC Driver           | Middleware between app and SQL                             | Version management, driver compatibility |
| Configuration Manager | UI tool to manage settings                                 | Centralized control over connectivity    |

**Visual diagram** showing the *SQL Native Client Configuration architecture (Protocols + Aliases + Flow)*



## SQL Server Browser Service — what it is, how it works, why it's important, and how DBAs deal with it in production environments.

### What is SQL Server Browser?

**SQL Server Browser** is a **Windows service** that helps client applications **connect to the correct SQL Server instance and port number** — especially when multiple instances exist on a single machine.

#### Executable file:

sqlbrowser.exe

#### Service name:

SQL Browser

#### Default startup type:

- **Disabled** (in some security-focused setups)
- **Manual** or **Automatic** (in multi-instance setups)

### How it Works (Step-by-Step)

When a client (like SSMS, app, or ODBC) tries to connect:

1. The **client sends a UDP packet** to port **1434** on the target SQL Server machine.
2. The **SQL Browser service** listens on **UDP port 1434**.
3. Browser reads the local **registry** to find:
  - Instance names
  - Corresponding TCP ports or named pipe paths
4. It **replies** to the client with the port number or pipe name for that instance.
5. The client then connects **directly to that TCP port**.

### Example Scenarios

#### Scenario 1 — Multiple Named Instances

You have a server named DBSERVER01 with:

- **Default instance** → MSSQLSERVER → Port 1433
- **Named instance** → SQLDEV → Dynamic port 51432

If a client connects using:

DBSERVER01\SQLDEV

- The client doesn't know the port number.
- It sends a UDP query to port 1434.
- **SQL Browser** responds: "Instance SQLDEV is on port 51432."
- The client connects successfully.

If **SQL Browser is stopped**, the connection fails with:

A network-related or instance-specific error occurred while establishing a connection...

unless you explicitly specify:

DBSERVER01,51432

#### Scenario 2 — Dynamic vs Static Ports

- Named instances **use dynamic ports** by default.
- Each time SQL Server restarts, it may get a different port.
- SQL Browser makes it **possible to locate** those changing ports dynamically.

#### Best practice:

For production, DBAs often **assign a static port** (e.g., 1500, 1501) and **disable Browser** for tighter security.

### ✖ Scenario 3 — Firewalls and Security Zones

If a firewall blocks UDP 1434:

- SQL Browser can't respond.
- Client connections to named instances fail.
- DBA needs to **open UDP 1434** or configure **static ports** and connect explicitly.

### ✖ Scenario 4 — Clustered Environments

In **Failover Clusters** or **Always On Availability Groups**:

- SQL Browser runs **on each node**.
- It helps redirect clients to the active node/instance name.
- Without it, multi-instance clusters can be painful to manage.

### ✖ Scenario 5 — Application Connection Issues

You may see errors like:

SQL Network Interfaces, error: 26 - Error Locating Server/Instance Specified

This usually means:

- SQL Browser is not running
- UDP 1434 is blocked
- The client can't resolve the instance name → port mapping

### 🔒 Security Considerations

| Area                  | Recommendation                                      |
|-----------------------|-----------------------------------------------------|
| Port exposure         | UDP 1434 should not be open to the public internet. |
| Production hardening  | Use static ports and disable Browser if possible.   |
| Multi-instance server | Keep Browser enabled.                               |
| Firewall setup        | Allow inbound UDP 1434 (only from trusted subnets). |

### 🔧 DBA Commands & Tools

#### ✖ Check SQL Browser service status (PowerShell)

```
Get-Service -Name SQLBrowser
```

#### ✖ Start or Stop service

```
Start-Service -Name SQLBrowser
```

```
Stop-Service -Name SQLBrowser
```

#### ✖ Change startup type

```
Set-Service -Name SQLBrowser -StartupType Automatic
```

#### ✖ Check running ports

```
EXEC xp_readerrorlog 0, 1, 'listening';
```

### 📋 Real-Time Troubleshooting Checklist

| Problem                                               | Root Cause                              | Fix                               |
|-------------------------------------------------------|-----------------------------------------|-----------------------------------|
| "Error 26 - Error Locating Server/Instance Specified" | SQL Browser stopped or UDP 1434 blocked | Start SQL Browser / Open UDP 1434 |
| Can connect only using port number                    | SQL Browser not responding              | Restart Browser service           |
| Dynamic port changes on restart                       | Instance using dynamic ports            | Assign static port                |
| Works locally but not remotely                        | Firewall rules missing                  | Allow UDP 1434 + TCP port         |

## 🌿 Real-Time DBA Example

### Example:

You have two instances:

- MSSQLSERVER (default) on port 1433
- SQL2019DEV (named) on port 49172

### Scenario:

A developer tries to connect from another machine to SRV-DB01\SQL2019DEV and gets **Error 26**.

### Diagnosis:

1. SQL Browser was stopped.
2. UDP 1434 blocked in Windows Firewall.

### Fix:

Start-Service SQLBrowser

New-NetFirewallRule -DisplayName "Allow SQL Browser UDP" -Direction Inbound -Protocol UDP -LocalPort 1434 -Action Allow

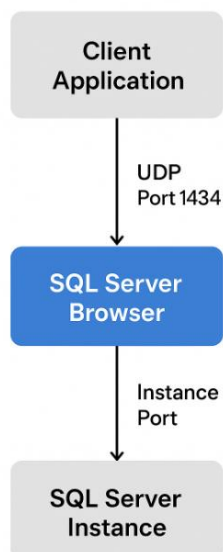


Connection restored.

### 💡 Summary

| Feature              | Description                                     |
|----------------------|-------------------------------------------------|
| Purpose              | Helps clients find instance ports dynamically   |
| Port                 | UDP 1434                                        |
| Needed for           | Named instances (especially with dynamic ports) |
| Can be disabled      | Yes, if using static ports                      |
| Security risk        | Minor exposure; limit scope in firewalls        |
| Critical in clusters | Yes, for multi-instance node discovery          |

### Flow (client → SQL Browser → instance port mapping):



## ◆ 1. What Are Protocols in SQL Server Configuration Manager?

SQL Server uses **network protocols** to communicate between **SQL Server (the database engine)** and **client applications** (like SSMS, applications, linked servers, etc.).

These protocols define **how data is transmitted** over the network between client and server.

In **SQL Server Configuration Manager → SQL Server Network Configuration → Protocols for** , you'll find:

| Protocol                               | Description                                                                               | Typical Usage                                               |
|----------------------------------------|-------------------------------------------------------------------------------------------|-------------------------------------------------------------|
| <b>Shared Memory</b>                   | Communication within the same machine (local connections only).                           | Local testing, when client and server are on same host.     |
| <b>Named Pipes</b>                     | Uses SMB (Server Message Block) for communication. Like file sharing, slower than TCP/IP. | Legacy applications, older environments, or local intranet. |
| <b>TCP/IP</b>                          | Uses TCP sockets for communication. The most common and recommended protocol.             | Client connections across networks (LAN/WAN/VPN).           |
| <b>VIA (Virtual Interface Adapter)</b> | Hardware protocol for specialized SAN/InfiniBand hardware.                                | <b>Deprecated</b> — do not use.                             |

## ◆ 2. Where to Find Them

Open **SQL Server Configuration Manager →**

SQL Server Network Configuration → Protocols for <InstanceName>

You'll see:

- Shared Memory
- Named Pipes
- TCP/IP
- VIA (if applicable)

Each can be **Enabled/Disabled** and have **Properties** like port numbers or pipe names.

## ◆ 3. Detailed Explanation of Each Protocol

### ◆ (A) Shared Memory

- Fastest method — no network layer.
- Only works **locally** (same machine).
- Useful for testing when SQL client and SQL Server are same host.
- No configuration needed.
- Default protocol for local connections (e.g. .\SQLEXPRESS).

**Example Use Case:**

- Developer running SSMS on the same machine as SQL Server.

**Best Practice:**

- Keep **enabled** (default).
- If you have strict security policies, you can disable it in production servers to avoid local exploit vectors.

### ◆ (B) Named Pipes

- Communication via **named pipe files** using SMB protocol (\\.\pipe\sql\query).
- Works both locally and over LAN.
- Slightly slower than TCP/IP.
- May require extra firewall rules and permissions.

**Example Use Case:**

- Legacy applications (pre-SQL Server 2005) or older Windows authentication scenarios.
- Intranets with no DNS resolution (pipes can still connect via UNC path).



**Best Practice:**

- Disable unless a legacy app **explicitly requires** it.
- Causes unnecessary overhead on modern systems.

**◆ (C) TCP/IP**

- Uses IP addresses and ports (default **1433**) for communication.
- Supports **static or dynamic ports**.
- Most widely used and **recommended** protocol.

**Common Properties:**

- **Enabled:** Yes/No
- **Listen All:** True/False
- **IP Addresses Tab:**
  - **Active:** Yes
  - **Enabled:** Yes
  - **TCP Port:** e.g., 1433 (static)
  - **TCP Dynamic Ports:** blank (if static port is used)

**Example Use Case:**

- Clients connect remotely (e.g., SSMS from another machine).
- Cloud or on-prem production deployments.

**Best Practice:**

- Always **enable** TCP/IP for production and client-server setups.
- Use **static ports** for consistency (especially in firewalled environments).
- Disable **Listen All** if you want to bind SQL to specific NICs only.
- Ensure **Windows Firewall** allows inbound TCP on that port.

**◆ (D) VIA (Virtual Interface Adapter)**

- Hardware-specific, for SAN or special interconnects.
- **Deprecated** after SQL Server 2012.
- Should always be **disabled** unless required for legacy specialized hardware.

**◆ 4. Real-Time Scenarios for SQL Server Protocols****✅ Scenario 1: Client Cannot Connect Remotely****Symptoms:**

SSMS error: "A network-related or instance-specific error occurred..."

**Possible Causes & Fixes:**

- TCP/IP disabled in SQL Configuration Manager → **Enable TCP/IP**.
- SQL Server Browser not running for named instance → **Start SQL Browser**.
- Firewall blocking port 1433 → **Allow inbound TCP 1433**.
- Static port misconfiguration → **Check IPAll → TCP Port**.

**✅ Scenario 2: Application Fails After Migration****Symptoms:**

Application used Named Pipes before migration but new server only has TCP/IP enabled.

**Fix:**

- Either **enable Named Pipes** or **update connection string**:
- np:\\.pipe\MSSQL\$InstanceName\sql\query

to

tcp:<servername>,1433

### ✓ Scenario 3: Port Conflict / Multiple Instances

- Multiple instances on same machine → each must use different TCP port.
- Configure static ports manually:
  - MSSQLSERVER → 1433
  - INST1 → 1434
  - INST2 → 1444

#### Fix:

Edit “IPAll → TCP Port” in Configuration Manager.

### ✓ Scenario 4: Security Hardening

- Disable all unused protocols (e.g., Named Pipes, VIA).
- Use **firewall + static ports**.
- Bind only to internal NICs if needed (uncheck “Listen All”).

### ✓ Scenario 5: Performance Troubleshooting

- Shared Memory vs TCP/IP: if performance differs drastically, check protocol order in **SQL Native Client Configuration → Client Protocols**.
- Ensure TCP/IP is **top of the order** for remote clients.

## ◆ 5. Client Protocols (Bonus)

You also have:

### SQL Native Client Configuration → Client Protocols

It defines what protocols the **client** will attempt to use.

Example:

Order:

1. TCP/IP
2. Named Pipes
3. Shared Memory

Best practice:

Keep **TCP/IP first** (most reliable and network-efficient).

## ◆ 6. Best Practices Summary

### Area                      Best Practice

**Enablement** Enable only **Shared Memory** (for local) and **TCP/IP** (for remote). Disable others.

**TCP Port** Use **static port 1433** for default instance, different static ports for named instances.

**Firewall** Allow inbound TCP on configured port(s).

**SQL Browser** Enable only if using named instances or dynamic ports.

**Security** Disable unneeded protocols (Named Pipes, VIA).

**Performance** Use TCP/IP as default, set proper NIC bindings.

**Monitoring** Track connection protocols via DMV:

```
SELECT protocol_desc, local_net_address, client_net_address
FROM sys.dm_exec_connections;
```

## ◆ 7. Verification Query (For DBAs)

To check which protocol a connection is using:

```
SELECT
    session_id,
    protocol_desc,
    client_net_address,
    local_net_address
FROM sys.dm_exec_connections
WHERE session_id = @@SPID;
```

Output example:

```
session_id | protocol_desc | client_net_address | local_net_address
```

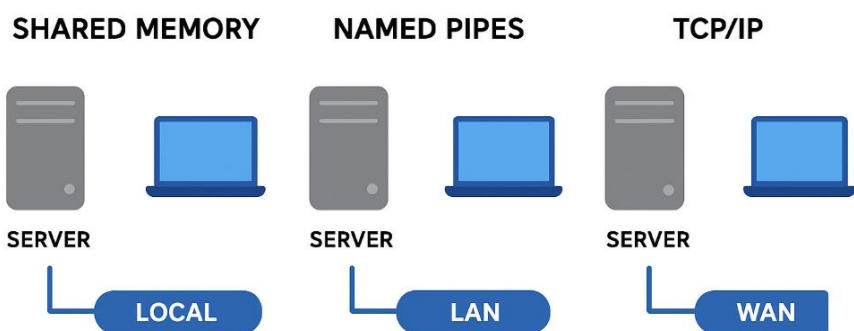
```
-----
52         | TCP          | 10.0.2.25         | 10.0.2.12
```

## ◆ 8. Real-Time DBA Checklist

- ✓ Enable TCP/IP
- ✓ Set static port
- ✓ Disable VIA and Named Pipes (if not needed)
- ✓ Allow port in firewall
- ✓ Restart SQL Service after protocol changes
- ✓ Test with telnet <hostname> 1433 or sqlcmd -S tcp:<hostname>,1433
- ✓ Verify with sys.dm\_exec\_connections

**Visual diagram:** SQL Server protocol (Shared Memory, Named Pipes, TCP/IP) works and where it's used (local, LAN, WAN)

# SQL SERVER PROTOCOLS



<https://www.sqldbachamps.com>

<https://github.com/PMSQLDBA/PraveenMadupu>

Telegram Group → SQLDBACloud-DevOps <https://t.me/+r2OYsT2qRZJkYWVI>

Praveen Madupu

Mb: +91 98661 30093

Database Administrator